Q1. Big Data refers to extremely large and complex datasets that cannot be processed using traditional data processing systems.
It involves high volume, velocity, and variety of data.
Technologies like Hadoop and Spark are used to handle it efficiently.

Q2. MapReduce is a programming model used in Hadoop for processing large datasets in parallel.
The Map phase breaks data into smaller parts and processes them.
The Reduce phase combines the results to give the final output.

Q3. The 5 V's of Big Data are Volume, Velocity, Variety, Veracity, and Value.
Volume refers to the amount of data, and Velocity is the speed of data processing.
Variety means different data types, Veracity is accuracy, and Value is usefulness.

Q4. Big Data is classified into Structured, Semi-structured, and Unstructured data.
Structured data is organized in tables like databases.
Unstructured data includes images, videos, and text, while semi-structured lies in between.

Q5. Hadoop is an open-source framework used for distributed storage and processing of Big Data.
It uses HDFS for storage and MapReduce for processing.
It allows data to be handled across multiple systems efficiently.

Q6. **Introduction:**
HDFS (Hadoop Distributed File System) is a distributed storage system designed to handle very large datasets across multiple machines. It is a core component of Hadoop and is built for scalability, fault tolerance, and high throughput.

Architecture Overview:
HDFS follows a **master-slave architecture**, where one main node controls the system and several worker nodes store data.

- The **NameNode** acts as the master.
- The **DataNodes** act as slaves.

The NameNode is responsible for managing metadata such as file names, permissions, and block locations. It does not store actual data but keeps track of where each block is stored in the cluster.

On the other hand, DataNodes are responsible for storing the actual data. Files are divided into large blocks (usually 128MB), and these blocks are distributed across different DataNodes.

Working Process:
When a client wants to read or write a file, it first contacts the NameNode to get

metadata information. After that, the client directly communicates with DataNodes to perform operations. This reduces load on the NameNode and improves performance.

Key Features of HDFS Architecture:

- **Block Storage:** Files are split into blocks and stored across nodes.
- **Replication:** Each block is stored multiple times (default is 3).
- **Fault Tolerance:** If one node fails, data is still available from other nodes.

HDFS also uses a **heartbeat mechanism**, where DataNodes regularly send signals to the NameNode. If a heartbeat is missing, the NameNode assumes the node has failed.

Additional Components:
The **Secondary NameNode** helps in managing metadata by periodically merging log files. It is not a backup but supports the main NameNode.

Conclusion:
Overall, HDFS architecture is highly reliable and scalable. It is designed to store massive data efficiently while ensuring fault tolerance and data availability.

Q7. **Introduction:**
Replication is one of the most important features of HDFS. It ensures that data is not lost and is always available, even in case of failures.

Why Replication is Important:
In HDFS, each data block is stored multiple times across different DataNodes. The default replication factor is 3.

This means:

- Each block has three copies
- Stored in different machines (and sometimes racks)

Replication provides several benefits:

- **Fault Tolerance:** If one node fails, data is still available
- **High Availability:** Users can access data anytime
- **Improved Performance:** Multiple copies allow parallel access

Failure Handling in HDFS:
HDFS uses a heartbeat mechanism to detect failures. DataNodes send regular heartbeat signals to the NameNode.

If a DataNode fails:

- NameNode stops receiving heartbeat

- It marks the node as dead

After detecting failure, the NameNode checks which blocks have lost replicas.

Then:

- It identifies under-replicated blocks
- Creates new copies on other DataNodes

This process is automatic and ensures that the replication factor is maintained.

Data Access During Failure:
Clients are automatically redirected to other available replicas. So even if one node fails, users can still access data without interruption.

Conclusion:
Thus, replication ensures reliability, fault tolerance, and continuous availability of data in HDFS. It is a critical feature for handling large-scale distributed systems.